

Chapitre 1: Introduction à l'analyse des réseaux sociaux

Master: Sciences de données pour une industrie intelligente

Pr. Elyazid AKACHAR

Email: elyazid.akachar@gmail.com

- ➊ Définition d'un réseau social
- ➋ Analyse des réseaux sociaux
- ➌ Type de graphes pour représenter les réseaux sociaux
- ➍ Réseau non orienté
- ➎ Réseau orienté
- ➏ Caractéristiques d'un graphe
- ➐ Représentations des graphes
- ➑ Mesures de centralité des nœuds
- ➒ Python pour manipuler les graphes

Définition d'un réseau social

✍ Un **réseau social** est un ensemble d'acteurs, comprenant des individus, des groupes ou des organisations, qui sont reliés par des relations sociales. Ces relations peuvent inclure l'amitié, la collaboration, l'échange d'informations, l'influence ou tout autre type d'interaction.

✍ Les réseaux sociaux permettent ainsi de comprendre comment les acteurs se connectent, communiquent et interagissent au sein d'une communauté, qu'elle soit physique ou virtuelle.

✍ Les réseaux sociaux peuvent prendre différentes formes :

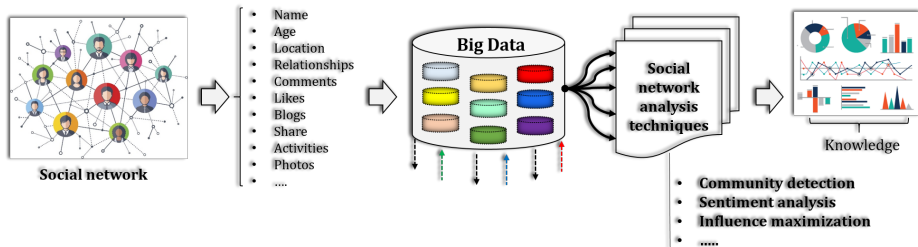
- Dans la vie réelle, par exemple au sein de la famille, des collègues, des associations, des écoles ou des universités.
- Sur les plateformes en ligne, telles que Facebook, LinkedIn, Twitter, etc.



Analyse des réseaux sociaux

✍ L'analyse des réseaux sociaux (**Social Network Analysis – SNA**) est un domaine de l'informatique qui vise à étudier les réseaux sociaux, en particulier les réseaux en ligne.

- Elle permet de modéliser et analyser les relations entre acteurs (individus, groupes, organisations) pour comprendre la structure, la dynamique et l'influence au sein des réseaux sociaux.
- La SNA fournit des outils pour visualiser les réseaux, calculer des indicateurs comme la centralité ou la densité, et détecter des communautés ou des acteurs clés.



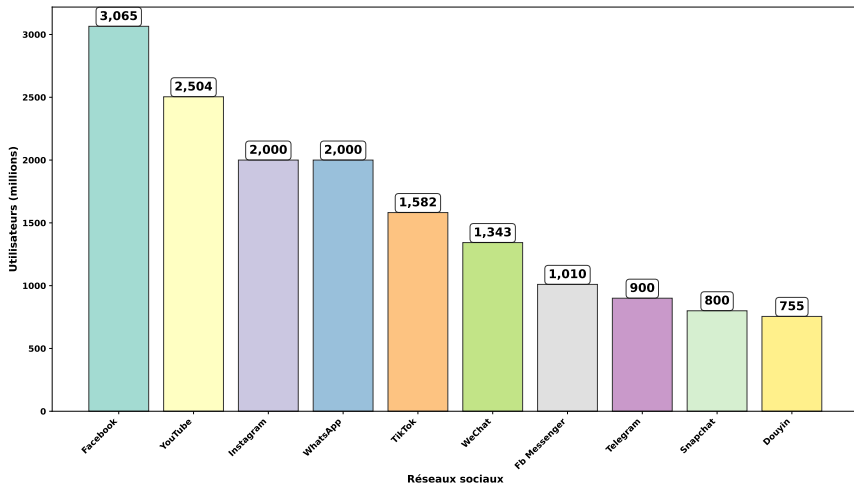
✍ L'analyse des réseaux sociaux comprend plusieurs sous-domaines spécialisés qui s'intéressent à différents aspects des réseaux. Voici les principaux :

- Community Detection
- Influence Maximization
- Link Prediction
- Change Point Detection
- Sentiment Analysis
- Node Classification
- Graph Embedding
- Anomaly Detection
- Information Diffusion
- Network Evolution
- Recommender Systems
- Fake News Detection
- Topic Modeling

Analyse des réseaux sociaux

📁 L'analyse des réseaux sociaux (SNA) constitue l'un des domaines de recherche les plus actifs et les plus explorés en informatique, en raison de la croissance rapide des réseaux en ligne et des données sociales.

📁 Classement des 10 réseaux sociaux les plus populaires dans le monde en janvier 2025, selon le nombre d'utilisateurs actifs (source : www.statista.com)



Analyse des réseaux sociaux

✍ Les réseaux sociaux peuvent être analysés à partir de trois types de variables principales :

- **Variables structurelles (Structural variables)** : elles représentent des informations sur les relations entre tous les acteurs du réseau.
- **Variables de composition (Composition variables)** : elles contiennent des informations caractéristiques sur les acteurs eux-mêmes, par exemple le nom, l'âge, le sexe, ou toute autre donnée individuelle.
- **Variables d'affiliation (Affiliation variables)** : elles décrivent les acteurs en fonction de leur appartenance à différents groupes, clubs, organisations ou communautés.

✍ En informatique, l'analyse des réseaux sociaux repose sur leur représentation sous forme de **graphes**, ce qui permet d'étudier plus facilement les relations et les interactions entre les acteurs.



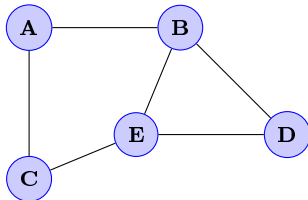
✎ Un graphe est une structure mathématique utilisée pour modéliser des relations entre différents éléments appelés nœuds (ou sommets). Ces relations sont représentées par des liens (ou arêtes) reliant les nœuds entre eux.

Un graphe est défini comme un couple :

$$G = (V, E)$$

où :

- V est l'ensemble des sommets (ou nœuds), représentant les acteurs du système.
- E est l'ensemble des arêtes (ou liens), représentant les relations ou interactions entre ces acteurs.
- Chaque arête relie deux sommets : $E \subseteq V \times V$



Exemple d'un graphe avec 5 nœuds et 6 arêtes

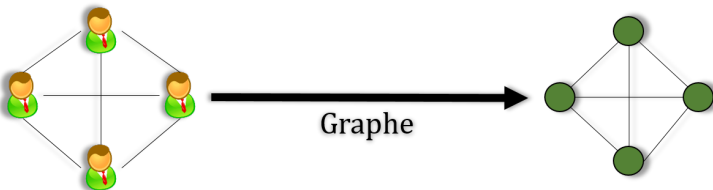
- $E = \{A, B, C, D, E\}$
- $V = \{(A, B), (A, C), (B, E), (B, D), (C, E), (E, D)\}$

Analyse des réseaux sociaux

✍ Dans le cadre de l'analyse des réseaux sociaux, les nœuds représentent les acteurs (individus, organisations, institutions, etc.) et les arêtes représentent les interactions ou les relations entre ces acteurs (amitié, communication, collaboration, suivi, etc.).

Acteurs (utilisateurs, individus) $\Rightarrow$ Nœuds (sommets, vertex)

Interactions (relations) $\Rightarrow$ Arêtes (liens)



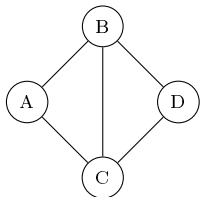
✍ Puisqu'il existe plusieurs réseaux sociaux, est-il possible ou efficace de modéliser tous les réseaux avec le même type de graphe ?

Type de graphes pour représenter les réseaux sociaux

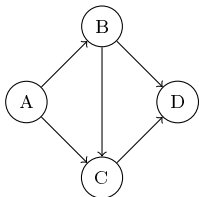
✍️ Puisque la modélisation des réseaux sociaux repose principalement sur la théorie des graphes et que ce domaine comprend plusieurs types de graphes, il est nécessaire de choisir le type de graphe le plus approprié pour chaque réseau social.

✍️ Les types de graphes les plus couramment utilisés en analyse des réseaux sociaux sont :

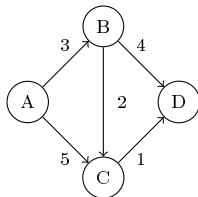
- **Graphe non orienté (Undirected Graph)** : relations réciproques, par exemple les amitiés sur Facebook.
- **Graphe orienté (Directed Graph / Digraph)** : relations avec un sens, par exemple les abonnements sur Twitter.
- **Graphe pondéré (Weighted Graph)** : relations avec intensité ou fréquence, par exemple le nombre de messages échangés entre deux utilisateurs.
- **Graphe biparti (Bipartite Graph)** : relations entre deux ensembles distincts de nœuds, par exemple utilisateurs et groupes ou événements.



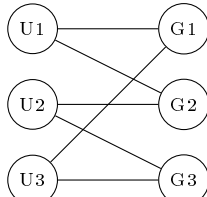
Undirected Graph



Directed Graph



Weighted Graph



Bipartite Graph

Graphe non orienté

Un **graphe non orienté** $G = (V, E)$ est une structure mathématique composée :

- d'un ensemble fini et non vide de **sommets** V , représentant les entités du réseau ;
- et d'un ensemble d'**arêtes** $E = \{\{u, v\} | u, v \in V, u \neq v\}$, représentant les connexions entre ces sommets.

Dans un graphe non orienté, les arêtes n'ont pas de direction. Ainsi, si deux sommets $(A, B) \in E$, alors la relation est réciproque :

$$(A, B) = (B, A)$$

Autrement dit, le lien entre A et B est **symétrique** : il n'existe pas de sens privilégié dans la relation.

- Deux sommets v_i et v_j sont dits adjacents ou voisins s'ils sont reliés par une arête $\{v_i, v_j\} \in E$.

✎ Caractéristiques principales :

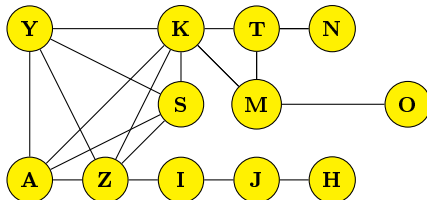
- Les liens sont symétriques : si A est connecté à B , alors B est automatiquement connecté à A .
- Les arêtes sont représentées par des segments simples (sans flèches).
- Ce type de graphe est souvent utilisé pour modéliser les relations mutuelles ou bidirectionnelles.

Exemples dans les réseaux sociaux :

- Facebook  : si deux utilisateurs sont amis, la relation est réciproque.
- LinkedIn  (connexion mutuelle) : deux personnes qui acceptent une relation professionnelle.

Modélisation d'un réseau social (Facebook) : On considère un extrait simplifié d'un réseau social composé de 12 utilisateurs : $V = \{A, S, K, Y, Z, T, M, I, J, H, N, O\}$. Les relations d'amitié entre ces utilisateurs sont les suivantes :

- A est ami avec S, K, Y et Z.
- S est amie avec A, K, Y et Z.
- K est ami avec A, S, Y, Z, T et M.
- Y est ami avec A, S, K et Z.
- Z est amie avec A, S, K, Y et I.
- I est amie avec Z et J.
- J est ami avec I et H.
- H est ami avec J.
- T est ami avec K, N et M.
- N est amie avec T.
- O est ami avec M.
- M est amie avec K, T et O.



✎ Un graphe non orienté pondéré est un graphe non orienté dans lequel chaque arête est associée à un poids (ou valeur), qui représente généralement une force, un coût, une distance ou une intensité de la relation entre les nœuds.

Graphe non orienté pondéré

Un **graphe non orienté pondéré** est un triplet

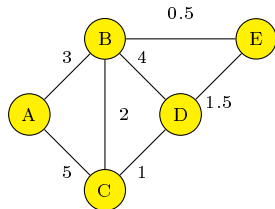
$$G = (V, E, w)$$

où :

- V est un ensemble **fini et non vide** de sommets (ou nœuds) ;
- $E \subseteq \{\{v_i, v_j\} \mid v_i, v_j \in V, v_i \neq v_j\}$ est un ensemble d'**arêtes** reliant les sommets de V ;
- $w : E \rightarrow \mathbb{R}$ est une **fonction de poids** qui associe à chaque arête $e \in E$ un **poids** $w(e)$.

✎ Les graphes non orientés pondérés sont utilisés pour modéliser des situations comme :

- distances entre villes,
- coûts dans un réseau de transport,
- intensité des interactions dans un réseau social.



Graphe orienté

Un **graphe orienté** $G = (V, E)$ est une structure mathématique composée :

- d'un ensemble **fini et non vide** de nœuds V , représentant les entités du réseau ;
- et d'un ensemble d'**arcs** $E \subseteq V \times V$, représentant les relations dirigées entre ces nœuds.

Chaque arc orienté $(v_i, v_j) \in E$ va du nœud v_i (source) vers le nœud v_j (cible) et est représenté graphiquement par $v_i \rightarrow v_j$.

- v_i est le sommet initial ou origine, et v_j est le sommet terminal ou extrémité.
- L'arc $a = (v_i, v_j)$ est sortant pour v_i et entrant ou incident pour v_j .
- Le sommet v_j est un **successeur** de v_i , tandis que v_i est un **prédécesseur** de v_j .
- Les arêtes ne sont pas réciproques : $(v_i, v_j) \in E$ n'implique pas $(v_j, v_i) \in E$.

✎ L'ensemble des successeurs d'un sommet $v_i \in V$ est noté :

$$\text{Succ}(v_i) = \{v_j \in V \mid (v_i, v_j) \in E\}$$

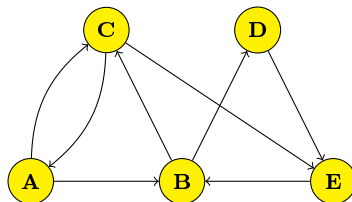
✎ L'ensemble des prédécesseurs d'un sommet $v_i \in V$ est noté :

$$\text{Pred}(v_i) = \{v_j \in V \mid (v_j, v_i) \in E\}$$

✍ Les graphes orientés sont utilisés pour modéliser des relations directionnelles, comme le suivi (follow) sur les réseaux sociaux, notamment Twitter (X) 🐦 et Instagram 📷.

✍ **Exemple** : On considère cinq utilisateurs A, B, C, D et E sur Instagram. Les relations “Follow” entre eux sont les suivantes :

- A suit B et C
- B suit C et D
- C suit A et E
- D suit E
- E suit B



Graphe orienté pondéré

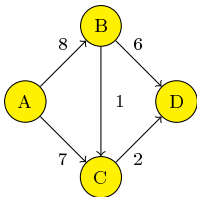
Un **graphe orienté pondéré** est défini comme un triplet

$$G = (V, A, w)$$

où :

- V est un ensemble fini et non vide de sommets ;
- $A \subseteq V \times V$ est un ensemble d'arcs orientés ;
- $w : A \rightarrow \mathbb{R}$ est une fonction de poids qui associe à chaque arc $(v_i, v_j) \in A$ une valeur réelle $w(v_i, v_j)$ représentant l'intensité, le coût ou la force de la relation orientée de v_i vers v_j .

🔗 Formellement, un arc orienté (v_i, v_j) de poids $w(v_i, v_j)$ indique une relation dirigée de v_i vers v_j avec une intensité mesurée par le poids associé.



🔗 Dans le contexte des réseaux sociaux, cela peut représenter :

- Le nombre de messages envoyés d'un utilisateur à un autre,
- Le niveau d'influence
- Le taux d'interaction (likes, commentaires, etc.).

Caractéristiques d'un graphe

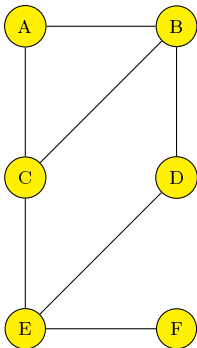
Chaîne et distance dans les graphes non orientés

✎ Une **chaîne** (ou *path* en anglais) est une suite de nœuds $(v_1, v_2, \dots, v_k)$ telle que chaque paire consécutive (v_i, v_{i+1}) est reliée par une arête.

✎ La direction n'a pas d'importance, car le graphe est non orienté.

✎ La distance entre deux nœuds est le nombre minimum d'arêtes dans une chaîne reliant ces nœuds.

✎ Pour les graphes non orientés pondéré, la distance entre deux sommets u et v est le poids total minimum parmi tous les chemins possibles allant de u à v .



✎ Considérons les nœuds A et D dans notre graphe.

✎ Il existe plusieurs chaînes entre A et D :

- $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D$ (longueur = 4 arêtes)
- $A \rightarrow C \rightarrow E \rightarrow D$ (longueur = 3 arêtes)
- $A \rightarrow B \rightarrow D$ (longueur = 2 arêtes)

✎ La **distance** entre A et D est la longueur minimale parmi toutes les chaînes reliant ces nœuds :

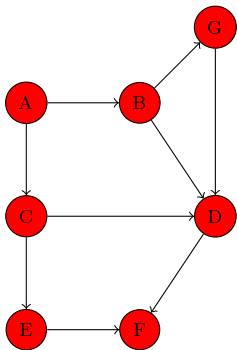
$$\text{dist}(A, D) = 2$$

✎ **Remarque** : la distance correspond toujours à la chaîne la plus courte, pas nécessairement à la première que l'on voit.

Caractéristiques d'un graphe

Chemin et distance dans les graphes orientés

- ✎ Un **chemin** (ou *directed path*) est une suite de nœuds $(v_1, v_2, \dots, v_k)$ telle que chaque arc suit la direction : $v_i \rightarrow v_{i+1}$.
- ✎ La **distance** entre deux nœuds est le nombre minimum d'arcs dans un chemin dirigé reliant ces nœuds.
- ✎ Si aucun chemin dirigé n'existe, la distance est ∞ .
- ✎ Dans un graphe orienté pondéré, la distance entre deux nœuds u et v est la somme minimale des poids parcourus sur les chemins dirigés de u à v .



✎ Considérons les nœuds A et F dans notre graphe orienté.

✎ Il existe plusieurs chemins entre A et F :

- $A \rightarrow B \rightarrow D \rightarrow F$ (longueur = 3 arêtes)
- $A \rightarrow C \rightarrow D \rightarrow F$ (longueur = 3 arêtes)
- $A \rightarrow C \rightarrow E \rightarrow F$ (longueur = 3 arêtes)
- $A \rightarrow B \rightarrow G \rightarrow D \rightarrow F$ (longueur = 4 arêtes)

✎ La **distance** entre A et F est la longueur minimale parmi tous les chemins reliant ces nœuds :

$$\text{dist}(A, F) = 3$$

✎ **Remarques** : La distance est **asymétrique** : par exemple, $\text{dist}(F, A) = \infty$ car aucun chemin dirigé ne relie F à A .

Représentations des graphes

✎ En informatique, un graphe doit être représenté de manière à ce que l'ordinateur puisse stocker et manipuler les informations sur les nœuds et les arêtes. Dans la plupart des cas, un graphe est représenté par : **matrice d'adjacence** ou **liste d'adjacence**.

❶ Matrice d'adjacence (Adjacency Matrix)

Matrice d'adjacence d'un graphe orienté ou non

Soit $G = (V, E)$ un graphe orienté ou non avec $|V| = n$ noeuds. La **matrice d'adjacence** de G est la matrice carrée A de taille $n \times n$, dont les coefficients $A[i][j]$, pour $0 \leq i \leq n - 1$ et $0 \leq j \leq n - 1$, sont définis par :

$$A[i][j] = \begin{cases} 1, & \text{si } (v_i, v_j) \in E, \\ 0, & \text{sinon.} \end{cases}$$

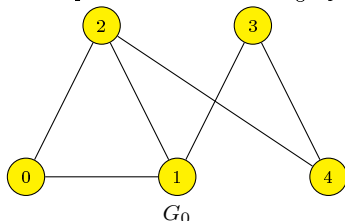
- Si G est **non orienté**, alors A est **symétrique** : $A[i][j] = A[j][i]$.
- Si G est **orienté**, A n'est **pas nécessairement symétrique** : $A[i][j] \neq A[j][i]$.

✎ Dans le cas d'un graphe pondéré $G = (V, E, w)$, on modifie légèrement la définition de A en attribuant à $A[i][j]$ une valeur différentes de toutes les autres. Par exemple la valeur ∞ lorsque $(v_i, v_j) \notin E$ et la valeur $w(v_i, v_j)$ si $(v_i, v_j) \in E$.

$$A[i][j] = \begin{cases} w(v_i, v_j), & \text{si } (v_i, v_j) \in E, \\ \infty, & \text{sinon.} \end{cases}$$

Représentations des graphes

 **Exemple :** on considère les graphes suivants :

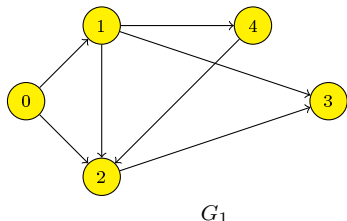


La matrice d'adjacence A de G_0 est :

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

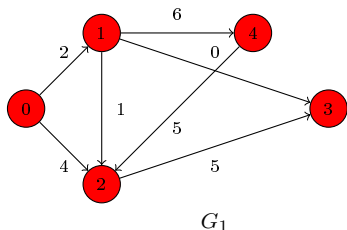
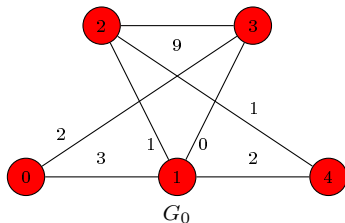
La matrice d'adjacence A de G_1 est :

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$



Représentations des graphes

 **Exemple** : on considère les graphes suivants :



La matrice d'adjacence A de G_0 est :

$$A = \begin{pmatrix} \infty & 3 & \infty & 2 & \infty \\ 3 & \infty & 1 & 0 & 2 \\ \infty & 1 & \infty & 9 & 1 \\ 2 & 0 & 9 & \infty & \infty \\ \infty & 2 & 1 & \infty & \infty \end{pmatrix}$$

La matrice d'adjacence A de G_1 est :

$$A = \begin{pmatrix} \infty & 2 & 4 & \infty & \infty \\ \infty & \infty & 1 & 0 & 6 \\ \infty & \infty & \infty & 5 & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 5 & \infty & \infty \end{pmatrix}$$

✎ Pour un graphe d'ordre n la complexité spatiale (càd la taille mémoire nécessaire) de l'implémentation de la matrice d'adjacence est en $O(n^2)$.

✎ La recherche d'un arc s'effectue en temps constant, cependant, pour déterminer si un sommet possède un voisin ou un successeur, il est nécessaire, dans le pire des cas, d'effectuer un parcours de la ligne correspondante de la matrice et la complexité est donc linéaire en $O(n^2)$.

✎ **Avantages de l'implémentation par matrice d'adjacence :**

- Représentation simple et directe.
- Utile pour vérifier rapidement si deux nœuds sont connectés.
- Pratique pour certains algorithmes matriciels (ex. calcul des chemins, centralités, etc.).

✎ **Inconvénients de l'implémentation par matrice d'adjacence :**

- Nécessite plus d'espace mémoire : n^2 cases, même si le graphe est sparse.
- Peu efficace pour les graphes avec beaucoup de nœuds mais peu d'arêtes.

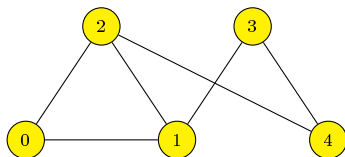
Représentations des graphes

② Liste d'adjacence (Adjacency list)

✎ Une liste d'adjacence est une autre manière de représenter un graphe, souvent plus économe en mémoire que la matrice d'adjacence, surtout lorsque le graphe est peu dense (c'est-à-dire qu'il contient peu d'arêtes par rapport au nombre maximal possible).

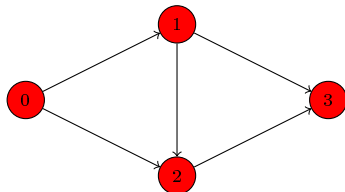
Liste d'adjacence d'un graphe orienté ou non

La liste d'adjacence d'un graphe $G = (V, E)$, orienté ou non, est une collection de listes où chaque sommet $v_i \in V$ est associé à la liste de ses nœuds adjacents (dans le cas d'un graphe non orienté) ou à la liste de ses successeurs (dans le cas d'un graphe orienté).



✎ Liste d'adjacence correspondante :

0	:	[1, 2]
1	:	[0, 2, 3, 4]
2	:	[0, 1, 4]
3	:	[1, 4]
4	:	[2, 3]



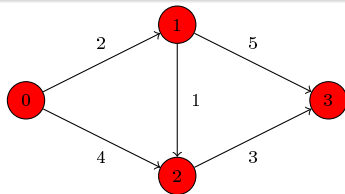
✎ Liste d'adjacence correspondante :

0	:	[1, 2]
1	:	[2, 3]
2	:	[3]
3	:	[]

Liste d'adjacence d'un graphe pondéré

Pour un graphe pondéré $G = (V, E)$, soit orienté ou non, la **liste d'adjacence** d'un sommet $v_i \in V$ est la collection des couples $(v_j, w(v_i, v_j))$ pour tous les voisins (ou successeurs) v_j de v_i , où $w(v_i, v_j)$ est le poids associé à l'arête (ou à l'arc) (v_i, v_j) :

$$\text{Adj}(v_i) = \{(v_j, w(v_i, v_j)) \mid (v_i, v_j) \in E\}$$



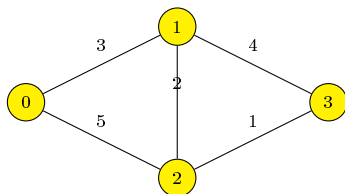
☞ Liste d'adjacence correspondante :

0 : [(1, 2), (2, 4)]

1 : [(2, 1), (3, 5)]

2 : [(3, 3)]

3 : []



☞ Liste d'adjacence correspondante :

0 : [(1, 3), (2, 5)]

1 : [(0, 3), (2, 2), (3, 4)]

2 : [(0, 5), (1, 2), (3, 1)]

3 : [(1, 4), (2, 1)]

Représentations des graphes

📌 dans la représentation par liste d'adjacence :

- Chaque sommet du graphe est représenté par une clé.
- Cette clé est associée à une liste contenant tous les sommets voisins (ou successeurs).
- Cette structure est donc souvent implémentée sous forme de dictionnaire ou table de hachage en informatique.

📌 **Avantages :**

- Efficace en mémoire pour les graphes peu denses : complexité spatiale $O(n + m)$, où $n = |V|$ et $m = |E|$.
- Permet d'accéder facilement aux voisins d'un sommet.

📌 **Inconvénients :**

- Quand presque tous les sommets sont reliés entre eux, la liste peut devenir longue et complexe à gérer.
- Certaines opérations, comme vérifier si deux sommets sont reliés ou calculer rapidement la densité globale du graphe, sont moins efficaces qu'avec une matrice.

📌 **On privilégie la matrice d'adjacence pour les graphes denses et les recherches rapides d'arcs, et la liste d'adjacence pour les graphes sparse afin de réduire l'utilisation mémoire.**

Mesures de centralité des nœuds

✍ La centralité d'un nœud dans un réseau social mesure son importance ou influence par rapport aux autres nœuds. Elle permet de détecter les acteurs clés dans un réseau et d'identifier ceux qui ont une position stratégique pour la diffusion d'information, la communication ou le contrôle des flux.

✍ Il existe plusieurs types de centralité, parmi les plus utilisées :

**Centralité de degré
(Degree Centrality)**

**Centralité de proximité
(Closeness Centrality)**

**Centralité d'intermédierité
(Betweenness Centrality)**

**Centralité d'influence
(Eigenvector Centrality)**

❶ Centralité de degré (Degree Centrality)

✍ La centralité de degré est l'une des mesures les plus simples et intuitives pour évaluer l'importance d'un nœud dans un réseau.

Centralité de degré pour un graphe non orienté

Pour un graphe non orienté $G = (V, E)$ de taille $|V| = n$, représenté par une matrice d'adjacence $A = (A[i][j])$, le **degré** d'un nœud v_i correspond au **nombre de ses voisins**, c'est-à-dire le nombre de nœuds **adjacents** à v_i . Il est calculé par la formule suivante :

$$d(v_i) = \sum_{j=1}^n A[i][j] \quad \begin{array}{l} \text{Formule normalisée} \\ \text{Centralité de degré} \end{array} \Rightarrow DC(v_i) = \frac{d(v_i)}{n-1} = \sum_{j=1}^n \frac{A[i][j]}{n-1}$$

✍ Plus un nœud a de voisins, plus il est central ou influent dans le réseau.

✍ Cette mesure est souvent utilisée pour identifier les utilisateurs ayant beaucoup d'amis dans les réseaux sociaux.

Centralité de degré pour un graphe orienté

Pour un graphe orienté $G = (V, E)$ de taille $|V| = n$, chaque nœud $v_i \in V$ possède deux degrés : le **degré sortant** (*out-degree*), qui correspond au nombre d'arcs sortant de v_i , et le **degré entrant** (*in-degree*), qui indique le nombre d'arcs entrant vers v_i .

$$d_{out}(v_i) = \sum_{j=1}^n A[i][j]$$

$$d_{in}(v_i) = \sum_{j=1}^n A[j][i]$$

Le degré totale d'un nœud v_i est : $d(v_i) = d_{out}(v_i) + d_{in}(v_i)$

La **centralité de degré** d'un nœud v_i peut être définie par la normalisation de ces valeurs :

$$DC_{out}(v_i) = \frac{d_{out}(v_i)}{n-1}, \quad DC_{in}(v_i) = \frac{d_{in}(v_i)}{n-1}, \quad DC(v_i) = \frac{d(v_i)}{2(n-1)} = \frac{d_{out}(v_i) + d_{in}(v_i)}{2(n-1)}$$

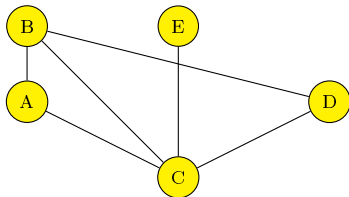
- $DC_{out}(v_i)$ (degré sortant) mesure l'influence potentielle de v_i : combien de nœuds il peut atteindre directement (nœud émetteur).
- $DC_{in}(v_i)$ (degré entrant) mesure la popularité de v_i : combien de nœuds sont connectés à lui (nœud récepteur).
- La somme $DC_{out}(v_i) + DC_{in}(v_i)$ donne le **degré total**, indiquant l'importance globale du nœud dans le réseau.

Mesures de centralité des nœuds

🔗 **Exemple : Centralité de degré pour un graphe non orienté**

Considérons le graphe $G = (V, E)$ avec :

$$V = \{A, B, C, D, E\}, \quad E = \{(A, B), (A, C), (B, C), (B, D), (C, D), (C, E), (C, E)\}$$



Matrice d'adjacence :

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Degré de chaque nœud :

$$d(A) = 2, \quad d(B) = 3, \quad d(C) = 4, \quad d(D) = 2, \quad d(E) = 1$$

Centralité de degré normalisée :

$$DC(v_i) = \frac{d(v_i)}{n-1} = \frac{d(v_i)}{4}$$

$$DC(A) = 0.5, \quad DC(B) = 0.75, \quad DC(C) = 1, \quad DC(D) = 0.5, \quad DC(E) = 0.25$$

Interprétation : Le nœud C est le plus central, B est également bien connecté, tandis que E est le moins central.

Mesures de centralité des nœuds

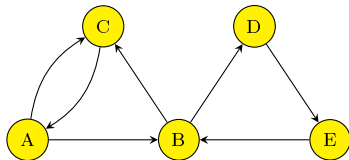
✎ Exemple : Centralité de degré pour un graphe orienté

Considérons le graphe orienté $G = (V, A)$ avec

$$V = \{A, B, C, D, E\}$$

et les arcs :

$$A \rightarrow B, A \rightarrow C, B \rightarrow C, B \rightarrow D, C \rightarrow A, D \rightarrow E, E \rightarrow B.$$



$$M = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

Degrés ($n = 5$) :

$$d_{out}(A) = 2, \quad d_{in}(A) = 1, \quad d(A) = 3,$$

$$d_{out}(B) = 2, \quad d_{in}(B) = 2, \quad d(B) = 4,$$

$$d_{out}(C) = 1, \quad d_{in}(C) = 2, \quad d(C) = 3,$$

$$d_{out}(D) = 1, \quad d_{in}(D) = 1, \quad d(D) = 2,$$

$$d_{out}(E) = 1, \quad d_{in}(E) = 1, \quad d(E) = 2.$$

Centralités de degré normalisées :

$$DC_{out}(v_i) = \frac{d_{out}(v_i)}{n-1}, \quad DC_{in}(v_i) = \frac{d_{in}(v_i)}{n-1}, \quad DC(v_i) = \frac{d_{out}(v_i) + d_{in}(v_i)}{2(n-1)}.$$

Avec $n-1 = 4$ et $2(n-1) = 8$:

$$\begin{aligned} DC_{out}(A) &= \frac{2}{4} = 0.50, & DC_{in}(A) &= \frac{1}{4} = 0.25, & DC(A) &= \frac{3}{8} = 0.375, \\ DC_{out}(B) &= 0.50, & DC_{in}(B) &= 0.50, & DC(B) &= \frac{4}{8} = 0.50, \\ DC_{out}(C) &= 0.25, & DC_{in}(C) &= 0.50, & DC(C) &= 0.375, \\ DC_{out}(D) &= 0.25, & DC_{in}(D) &= 0.25, & DC(D) &= 0.25, \\ DC_{out}(E) &= 0.25, & DC_{in}(E) &= 0.25, & DC(E) &= 0.25. \end{aligned}$$

Interprétation: B est le plus central (centralité totale 0.5) ; A et C ont une centralité intermédiaire ; D et E sont les moins centraux.

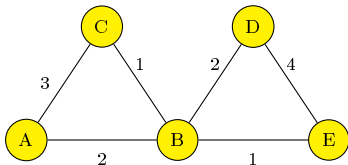
Degré pondéré (*Strength*) pour un graphe pondéré non orienté

Le **degré pondéré** (ou *strength*) d'un nœud v_i , dans un graphe pondéré non orienté $G = (V, E, W)$ de taille $n = |V|$, est défini comme la somme des poids des arêtes incidentes à ce nœud :

$$s(v_i) = \sum_{v_j \in V} w(v_i, v_j) \Rightarrow \text{formule normalisée : } DC^{(w)}(v_i) = \frac{s(v_i)}{(n-1) \cdot w_{\max}}$$

où $w(v_i, v_j)$ est le poids de l'arête reliant v_i et v_j .

- $s(v_i)$ mesure l'intensité totale des connexions du nœud v_i .
- Plus $s(v_i)$ est grand, plus le nœud est fortement connecté aux autres (par des liens de poids élevé).
- C'est une version pondérée du degré classique.



$$s(A) = 2 + 3 = 5$$

$$CD^{(w)}(A) = \frac{s(A)}{(n-1) \cdot w_{\max}} = \frac{5}{4 \cdot 4} = \frac{5}{16} \approx 0.3125$$

$$s(B) = 2 + 1 + 2 + 1 = 6, \quad CD^{(w)}(B) = 6/16 \approx 0.375$$

$$s(C) = 3 + 1 = 4, \quad CD^{(w)}(C) = 4/16 = 0.25$$

$$s(D) = 2 + 4 = 6, \quad CD^{(w)}(D) = 6/16 \approx 0.375$$

$$s(E) = 4 + 1 = 5, \quad CD^{(w)}(E) = 5/16 \approx 0.3125$$

Degré pondéré dans un graphe orienté

Dans un graphe orienté pondéré $G = (V, E, W)$ de taille $n = |V|$, chaque nœud v_i possède trois types de degré pondéré : le **degré sortant pondéré** $s_{out}(v_i)$, le **degré entrant pondéré** $s_{in}(v_i)$, et le **degré pondéré total** $s_T(v_i)$:

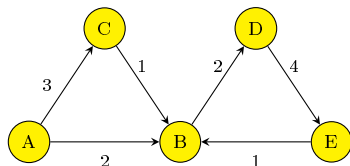
$$s_{out}(v_i) = \sum_{(v_i, v_j) \in E} w(v_i, v_j), \quad s_{in}(v_i) = \sum_{(v_j, v_i) \in E} w(v_j, v_i), \quad s_T(v_i) = s_{out}(v_i) + s_{in}(v_i)$$

Les versions normalisées sont données par :

$$DC_{out}^{(w)}(v_i) = \frac{s_{out}(v_i)}{(n-1)w_{\max}}, \quad DC_{in}^{(w)}(v_i) = \frac{s_{in}(v_i)}{(n-1)w_{\max}}, \quad DC_T^{(w)}(v_i) = \frac{s_T(v_i)}{2(n-1)w_{\max}}$$

où $w_{\max}$ représente le poids maximal des arcs du graphe.

- $s_{out}(v_i)$ représente l'activité d'un utilisateur (interactions qu'il envoie).
- $s_{in}(v_i)$ indique sa popularité (interactions reçues).
- $s_T(v_i)$ mesure son importance globale dans le réseau.



$$s_{out}(A) = 2 + 3 = 5, \quad s_{in}(A) = 0, \quad s_T(A) = 5 + 0 = 5$$

$$DC_{out}^{(w)}(A) = \frac{5}{(n-1)w_{\max}} = \frac{5}{4 \cdot 4} = \frac{5}{16} \approx 0.3125$$

$$DC_{in}^{(w)}(A) = 0, \quad DC_T^{(w)}(A) = \frac{5}{2 \times 4 \times 4} = \frac{5}{32} \approx 0.156$$

$$s_{out}(B) = 2, \quad s_{in}(B) = 4, \quad s_T(B) = 6, \quad DC_T^{(w)}(B) = \frac{6}{2 \times 4 \times 4} = \frac{6}{32} \approx 0.187$$

$$s_{out}(C) = 1, \quad s_{in}(C) = 3, \quad s_T(C) = 4, \quad DC_T^{(w)}(C) = \frac{4}{2 \times 4 \times 4} = \frac{4}{32} \approx 0.125$$

$$s_{out}(D) = 4, \quad s_{in}(D) = 2, \quad s_T(D) = 6, \quad DC_T^{(w)}(D) = \frac{6}{2 \times 4 \times 4} = \frac{6}{32} \approx 0.187$$

$$s_{out}(E) = 1, \quad s_{in}(E) = 4, \quad s_T(E) = 5, \quad DC_T^{(w)}(E) = \frac{5}{2 \times 4 \times 4} = \frac{5}{32} \approx 0.156$$

② Centralité de proximité (Closeness Centrality)

La centralité de proximité prend en compte tous les autres nœuds du graphe pour mesurer à quel point un nœud est central. Elle s'appuie sur la distance entre le nœud considéré et chacun des autres nœuds.

Centralité de proximité (Closeness Centrality)

La **centralité de proximité** d'un nœud v_i dans un graphe $G = (V, E)$ mesure à quel point ce nœud est proche de tous les autres nœuds du graphe. Elle est définie comme l'inverse de la somme des distances les plus courtes entre v_i et tous les autres nœuds :

$$C(v_i) = \frac{1}{\sum_{j \neq i} d(v_i, v_j)}, \quad \Rightarrow \text{formule normalisée :} \quad CC(v_i) = \frac{n-1}{\sum_{j \neq i} d(v_i, v_j)}$$

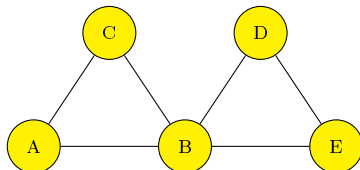
où $n = |V|$ est le nombre total de nœuds et $d(v_i, v_j)$ est la distance la plus courte entre v_i et v_j .

Plus $CC(v_i)$ est grand, plus le nœud v_i est central, c'est-à-dire proche de tous les autres nœuds du graphe. C'est une mesure globale de centralité.

Closeness centrality est une mesure globale contrairement au degré qui est locale.

Plus le CC est grand, plus le nœud est proche de tous les autres.

Dans les réseaux sociaux, il identifie les utilisateurs capables de diffuser ou de recevoir l'information rapidement.



Distances depuis A :

$$d(A, B) = 1, \quad d(A, C) = 1, \quad d(A, D) = 2, \quad d(A, E) = 2$$

Somme des distances :

$$\sum_{j \neq A} d(A, j) = 1 + 1 + 2 + 2 = 6$$

Closeness Centrality :

$$CC(A) = \frac{n-1}{\sum_{j \neq A} d(A, j)} = \frac{4}{6} \approx 0.667$$

Autres nœuds :

$$CC(B) = 0.8, \quad CC(C) = 0.667, \quad CC(D) = 0.571, \quad CC(E) = 0.571$$

✎ Selon le type de graphe, le calcul de la closeness centrality s'adapte :

✎ **Cas orienté :**

Dans un graphe orienté, la closeness centrality est calculée en tenant compte uniquement des chemins respectant la direction des arêtes, car la distance d'un nœud A vers B peut différer de celle de B vers A.

✎ **Cas pondéré:**

Dans un graphe pondéré, la closeness centrality est déterminée à partir des chemins les plus courts calculés selon les poids des arêtes, les poids représentant souvent le coût, la distance ou le temps de parcours entre les nœuds.

③ Centralité d'intermédiation (Betweenness Centrality)

✎ La Betweenness Centrality mesure à quel point un nœud joue un rôle d'intermédiaire dans un réseau. Autrement dit, elle indique combien de fois un nœud se trouve sur le chemin le plus court entre deux autres nœuds.

Betweenness centrality pour graphe non orienté

Pour un graphe $G = (V, E)$, la betweenness centrality de $v_i \in V$ est :

$$B(v_i) = \sum_{s \neq v_i \neq t} \frac{\sigma_{st}(v_i)}{\sigma_{st}} \Rightarrow \text{formule normalisée : } BC(v_i) = \frac{B(v_i)}{(n-1)(n-2)/2}$$

où :

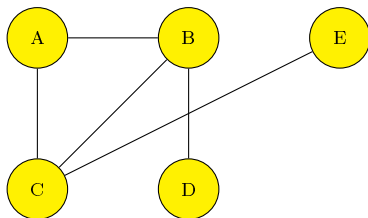
- σ_{st} = nombre de chemins les plus courts entre s et t
- $\sigma_{st}(v_i)$ = nombre de ces chemins passant par v_i

✎ Interprétation dans les réseaux sociaux

Les nœuds avec une betweenness élevée sont souvent :

- Des utilisateurs qui relient plusieurs communautés
- Des “influenceurs” ou points de passage clés pour l'information

Mesures de centralité des nœuds



Formule :

$$B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (\text{paires non ordonnées})$$

Calcul détaillé pour B :

Considérons les paires (s, t) parmi $\{A, C, D, E\}$ (6 paires) :

$$\{A, C\}, \{A, D\}, \{A, E\}, \{C, D\}, \{C, E\}, \{D, E\}.$$

- $\{A, C\}$: shortest path $A-C$ (longueur 1), $\sigma_{AC} = 1$, passe par B? non $\Rightarrow 0$.
- $\{A, D\}$: shortest path $A-B-D$ (longueur 2), $\sigma_{AD} = 1$, passe par B? oui $\Rightarrow 1$.
- $\{A, E\}$: shortest path $A-C-E$ (longueur 2), $\sigma_{AE} = 1$, passe par B? non $\Rightarrow 0$.
- $\{C, D\}$: shortest path $C-B-D$ (longueur 2), $\sigma_{CD} = 1$, passe par B? oui $\Rightarrow 1$.
- $\{C, E\}$: shortest path $C-E$ (longueur 1), $\sigma_{CE} = 1$, passe par B? non $\Rightarrow 0$.
- $\{D, E\}$: shortest path $D-B-C-E$ (longueur 3), $\sigma_{DE} = 1$, passe par B? oui $\Rightarrow 1$.

Somme des contributions :

$$B(B) = \frac{0}{1} + \frac{1}{1} + \frac{0}{1} + \frac{1}{1} + \frac{0}{1} + \frac{1}{1} = 3.$$

Normalisation (non orienté) : diviser par $(n-1)(n-2)/2$. Ici $n = 5$ donc facteur = 6.

$$BC(B) = \frac{3}{6} = 0.5.$$

🔖 Cas orienté ou pondéré

- Dans un graphe orienté, on ne compte que les chemins respectant la direction des arêtes.
- Dans un graphe pondéré, on utilise les chemins les plus courts en fonction du poids total minimal.
- La formule normalisée pour les graphes orientés est :

$$BC(v_i) = \frac{B(v_i)}{(n-1)(n-2)} \quad \text{où :} \quad B(v_i) = \sum_{s \neq v_i \neq t} \frac{\sigma_{st}(v_i)}{\sigma_{st}}$$

✍ Pour créer, analyser et visualiser des graphes et réseaux, on peut utiliser plusieurs packages Python :

- **NetworkX**
- **igraph**
- **Graph-tool**
- **PyGraphviz**
- **SNAP**
- **StellarGraph**

✍ NetworkX est une bibliothèque Python dédiée à la création, manipulation et analyse de graphes et réseaux.

✍ Elle permet de travailler avec des graphes orientés, non orientés et pondérés.

✍ **Fonctionnalités principales :**

- Création de graphes simples ou complexes.
- Calcul de mesures de centralité : degree, closeness, betweenness, etc.
- Recherche de plus courts chemins, cycles et composantes connexes.
- Analyse de communautés et sous-graphes.
- Visualisation simple avec Matplotlib ou PyGraphviz.

❶ Importation du module NetworkX

```
import networkx as nx
```

❷ Charger des graphes avec NetworkX

✍ NetworkX fournit plusieurs fonctions pour charger des graphes depuis différents formats, ce qui facilite l'importation et l'analyse des réseaux.

✍ Méthodes disponibles

- `nx.read_edgelist()` : charger depuis une edge list
- `nx.read_adjlist()` : charger depuis une adjacency list
- `nx.read_graphml()` : charger depuis un fichier GraphML
- `nx.read_gexf()` : charger depuis un fichier GEXF
- `nx.from_numpy_matrix()` : charger depuis une matrice d'adjacence

Python pour manipuler les graphes

```
import networkx as nx
import numpy as np

# 1. Edge list
G1 = nx.read_edgelist("graphe_edgelist.txt")

# 2. Adjacency list
G2 = nx.read_adjlist("graphe_adjlist.txt")

# 3. GraphML
G3 = nx.read_graphml("graphe.graphml")

# 4. GEXF
G4 = nx.read_gexf("graphe.gexf")

# 5. Matrice d'adjacence
mat = np.array([[0,1,0],
                [1,0,1],
                [0,1,0]])
G5 = nx.from_numpy_matrix(mat)
```

✍ NetworkX fournit aussi des graphes réels ou d'exemple, comme le graphe **Karate Club**, qui permettent de tester et illustrer des algorithmes sur des réseaux existants.

```
G = nx.karate_club_graph()
```

Python pour manipuler les graphes

③ Créer des graphes simples avec NetworkX

📁 Graphe non orienté

```
import networkx as nx
G = nx.Graph() # Création d'un graphe non orienté
G.add_nodes_from(["A", "B", "C", "D"]) # Ajouter plusieurs nœuds
G.add_edges_from([("A","B"), ("B","C"), ("C","D"), ("A","D")]) # Ajouter des arêtes
```

📁 Graphe orienté

```
DG = nx.DiGraph() # Création d'un graphe orienté
DG.add_edges_from([("A","B"), ("B","C"), ("C","A")])
```

📁 Graphe pondéré

```
WG = nx.Graph()
WG.add_edge("A","B", weight=2.5) # Ajouter une arête avec un poids
WG.add_edge("B","C", weight=1.0)
```

📁 Visualisation simple

```
import matplotlib.pyplot as plt
nx.draw(G, with_labels=True, node_color='lightblue', edge_color='gray')
plt.show()
```

③ Afficher les informations sur un graphe avec NetworkX

```
import networkx as nx
G = nx.karate_club_graph() # Exemple de graphe réel
print(nx.info(G)) # les informations générale
print(G.number_of_nodes()) # Affiche le nombre de nœuds
print(G.number_of_edges()) # Affiche le nombre d'arêtes
```

📄 Liste des nœuds

```
print(G.nodes())
```

📄 Liste des arêtes

```
print(G.edges())
```

📄 Attributs des nœuds

```
print(G.nodes(data=True))
```

📄 Attributs des arêtes

```
print(G.edges(data=True))
```

④ Distance entre les nœuds dans un graphe avec NetworkX

🔗 Distance entre deux nœuds

```
import networkx as nx
G = nx.karate_club_graph()
distance = nx.shortest_path_length(G, source="A", target="B")
print(distance) # Distance minimale entre A et B
```

🔗 Plus courts chemins

```
paths = nx.shortest_path(G, source="A", target="B")
print(paths) # Liste des nœuds sur le plus court chemin
```

🔗 Remarques :

- **nx.shortest_path_length()** : renvoie la distance minimale (nombre d'arêtes) entre deux nœuds.
- **nx.shortest_path()** : renvoie la séquence de nœuds constituant le plus court chemin.
- Pour un graphe pondéré, on peut ajouter **weight="weight"** dans ces fonctions pour tenir compte des poids des arêtes.

⑤ Centralités dans un graphe avec NetworkX

🔗 Degree Centrality (degré)

```
import networkx as nx
G = nx.karate_club_graph()
deg centrality = nx.degree centrality(G)
print(deg centrality)
```

🔗 Closeness Centrality (proximité)

```
closeness = nx.closeness centrality(G)
print(closeness)
```

🔗 Betweenness Centrality (intermédiation)

```
betweenness = nx.betweenness centrality(G)
print(betweenness)
```

🔗 Eigenvector Centrality (importance par influence)

```
eigenvector = nx.eigenvector centrality(G)
print(eigenvector)
```

🔗 Pour un graphe orienté, certaines centralités peuvent être calculées avec :

G.in_degree centrality() et **G.out_degree centrality()**.

🔗 Pour un graphe pondéré, ajouter **weight="weight"** dans les fonctions pour tenir compte des poids.